Milestone 2: Detailed Project Outline

Stock Trader

Kartheek Gavini, Philip Lee, and Na Ni

We will create a private GitHub repository and share it with our Project Mentor/TA. The repository will be used for version control and collaboration, and will also allow staff and team members to review code quality. (Note: the guidelines describe forming groups of 4. We currently only have 3 members and have confirmed with course staff that grading criteria will reflect this).

1. Motivation / Problem Statement

Retail investors and students frequently want to answer questions like: "Which stocks moved most this week?", "Which industries have unusual trading activity?", "Which stocks should I invest in?" Current tools either provide charts without systematic comparisons, or news without structured analytics. Our **Stock Tracker** web application will integrate large U.S. equities price datasets with a ticker-tagged news/events dataset to support queryable exploration: time-window performance metrics, sector/industry rankings, OHLCV analysis, and (optionally) news-volume vs. return relationships. This project is designed around a relational schema that enables multi-table joins and complex aggregations and query optimization.

2. Featured Implementations

1. **Ticker search + company profile:** search by ticker/name; show sector, industry, exchange metadata.
2. **Historical price analytics:** OHLCV chart over a selected window; derived metrics (return, rolling averages, volatility proxy).
3. **Sector/industry leaderboard:** top performers and most active stocks by sector/industry; filters by window and metric.
4. **Compare tickers:** compare multiple stocks by relative performance and correlation over a chosen time window.
5. **News explorer (scope-controlled):** show news volume over time and recent articles for a selected ticker (based on ticker-tagged dataset).

3. Potential Featured Implementations

6. **Authentication (Milestone 5 dependent):** user login and persistent server-side watchlists.
7. **Watchlists + alerts:** threshold alerts on returns or volume spikes.
8. **News sentiment/topic tags:** if news dataset includes sentiment, expose filters; otherwise optional NLP-based add-on using a NLP package (will try to do depending on complexity).

4. Application Pages

We will implement multiple pages whose functionality is meaningfully different.

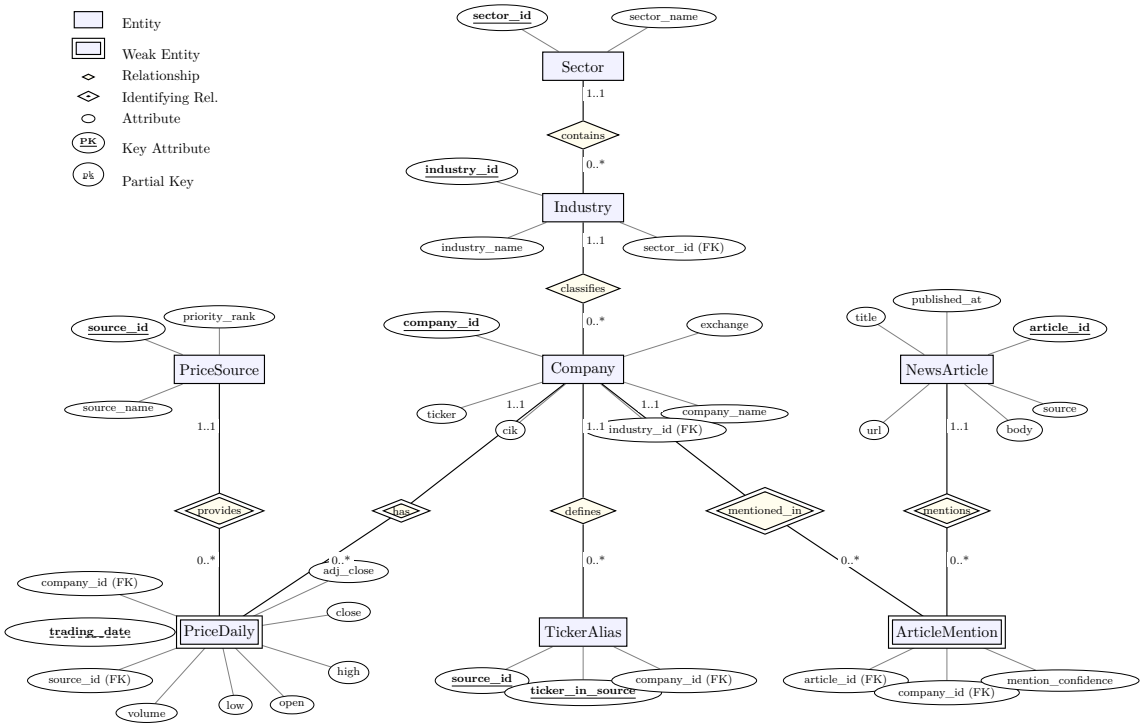
9. **Home / Market Overview:** top movers, most active by volume, and quick filters by sector/industry.
10. **Sector/Industry Page:** per-industry and per-sector rankings (top returns, volume spikes) with drilldown to tickers. An up to date price chart + derived metrics; recent news items and/or news volume trend for that ticker.
11. **Compare Page:** multi-ticker selection; correlation/relative performance table + chart for selected window.
12. **News Explorer Page:** browse/filter news by date/source; see most-mentioned tickers and click through.

5. Relational Schema — ER Diagram

We will model the domain using entities and relationships with clear cardinalities, then translate to relations using the standard approach: many–many relationships become separate tables, and relationship attributes belong on the relationship table

Core entities (first pass):

- Sector (1-to-many) Industry (1-to-many) Company
- Company (1-to-many) PriceDaily (fact table)
- NewsArticle (many-to-many) Company via Article dimension to support ingestion from multiple Kaggle price datasets and preserve provenance.



6. SQL DDL — Creating the Database

(DDL kept consistent with the ERD; includes provenance for multiple sources and a ticker-alias table for entity resolution.)

```
CREATE TABLE sector (
    sector_id    SERIAL PRIMARY KEY,
    sector_name  TEXT NOT NULL UNIQUE
);
CREATE TABLE industry (
    industry_id    SERIAL PRIMARY KEY,
    industry_name  TEXT NOT NULL,
    sector_id      INT NOT NULL REFERENCES sector(sector_id),
    UNIQUE (sector_id, industry_name)
);
CREATE TABLE company (
    company_id    SERIAL PRIMARY KEY,
    ticker        TEXT NOT NULL UNIQUE,
    company_name  TEXT,
    exchange      TEXT,
    cik           TEXT,
    industry_id   INT REFERENCES industry(industry_id)
);
```

```sql
CREATE TABLE price_source (
    source_id      SERIAL PRIMARY KEY,
    source_name    TEXT NOT NULL UNIQUE,
    priority_rank  INT NOT NULL
);

CREATE TABLE price_daily (
    company_id    INT NOT NULL REFERENCES company(company_id),
    trading_date  DATE NOT NULL,
    source_id     INT NOT NULL REFERENCES price_source(source_id),
    open          DOUBLE PRECISION,
    high          DOUBLE PRECISION,
    low           DOUBLE PRECISION,
    close         DOUBLE PRECISION,
    adj_close     DOUBLE PRECISION,
    volume        BIGINT,
    PRIMARY KEY (company_id, trading_date, source_id)
);

-- Entity resolution: map dataset-specific tickers to canonical
   company_id
CREATE TABLE ticker_alias (
    source_id         INT NOT NULL REFERENCES price_source(source_id),
    ticker_in_source  TEXT NOT NULL,
    company_id        INT NOT NULL REFERENCES company(company_id),
    PRIMARY KEY (source_id, ticker_in_source)
);

-- News
CREATE TABLE news_article (
    article_id    BIGSERIAL PRIMARY KEY,
    source        TEXT,
    published_at  TIMESTAMPTZ NOT NULL,
    title         TEXT NOT NULL,
    url           TEXT UNIQUE,
    body          TEXT
);

CREATE TABLE article_mention (
    article_id            BIGINT NOT NULL REFERENCES news_article(
       article_id)
                          ON DELETE CASCADE,
    company_id            INT NOT NULL REFERENCES company(company_id)
                          ON DELETE CASCADE,
    mention_confidence    DOUBLE PRECISION,
    PRIMARY KEY (article_id, company_id)
);

-- Indexes for expected workloads
CREATE INDEX idx_company_industry  ON company(industry_id);
CREATE INDEX idx_price_daily_date  ON price_daily(trading_date);
CREATE INDEX idx_news_published_at ON news_article(published_at);
CREATE INDEX idx_mention_company   ON article_mention(company_id);
```

7. Cleaning and Preprocessing Plan

We will standardize identifiers, normalize schemas across sources, and deduplicate carefully to support efficient queries and later entity resolution.

7.1. A) Canonical identifiers + entity resolution

- Create canonical company(company_id, ticker, ...) keyed by company_id.
- Normalize tickers (uppercase, trim, unify punctuation).
- Populate ticker_alias(source_id, ticker_in_source → company_id) for source-specific quirks.

7.2. B) Multi-source price ingestion

- For each Kaggle price dataset (e.g., "Stock Market Data" and "U.S. Stock Dataset"), load into staging, then normalize into price_daily.
- Use price_source.priority_rank to define a "preferred" source when duplicates exist.
- De-dup logic: if multiple sources provide the same (ticker, date), keep all rows (for provenance).

7.3. C) Sector + industry classification

- Even if your price datasets include labels, we will still maintain explicit sector and industry dimension tables for cleanliness and join performance.
- If classification differs across sources, choose a canonical mapping dataset (small) or define reconciliation rules (priority source, majority vote, etc.).

7.4. D) News ingestion

- Choose a ticker-tagged Kaggle news dataset (recommended: "Daily Financial News for 6000+ Stocks" or "News Headlines + Tickers") and ingest:
  - article fields → news_article
  - (article, ticker) links → article_mention
- If the dataset already provides tickers, article_mention is straightforward; if it provides company names, we map to tickers via canonical company table.

8. Web Application Tech Tools

We will build a three-tier client–server web app following the course's web architecture framing: React in the browser, Node/Express on the server, and PostgreSQL as the database.

8.1. Database / Hosting

- **PostgreSQL** hosted on **AWS RDS** (Milestone 3+). The server connects via the `pg` npm driver using the RDS endpoint on port 5432, with credentials stored in environment variables.
- Following the course's **Embedded SQL** approach, we write parameterized SQL directly in application code (no ORM), giving full control over joins, window functions, and aggregations.

8.2. Backend

- **Node.js + Express** for the API layer. Routes follow the `app.METHOD(PATH, HANDLER)` pattern; each handler builds a parameterized SQL query, executes it via `pool.query()`, and returns results with `res.json()`.
- Routes are organized into `express.Router()` modules: `/api/companies`, `/api/prices`, `/api/sectors`, `/api/compare`, and `/api/news`.
- Core dependencies: `express`, `pg`, `dotenv` (env-var loading), `cors` (cross-origin requests between React dev server and Express).

8.3. Frontend

- **React.js** scaffolded with `create-react-app`. All components are functional, using `useState` for local state and `useEffect` to trigger `fetch()` calls to the Express API on load or filter changes.
- **React Router** provides client-side routing across the four main pages (Home, Sector/Industry, Compare, News Explorer) without full page reloads.

8.4. Updates / Transactions

- For multi-step writes (e.g., watchlist updates), we wrap sequences in explicit transactions (`BEGIN ... COMMIT/ROLLBACK`). Default isolation is **READ COMMITTED**; we will only strengthen it if a specific feature requires it.

9. Team Responsibilities

Because we are currently a 3-person team, we will have to distribute more work per person and have each person take more responsibility for a subsection of the web application.

1. **Philip Lee (ETL, entity resolution and normalization):**
   Organize and normalize into price_daily; create/maintain canonical company, industry, sector, and ticker_alias; document cleaning rules.

2. **Na Ni (DB + SQL lead):**
   Finalize ERD + DDL; implement complex SQL queries (leaderboards, anomalies, correlation helpers); indexing strategy; performance testing plan.

3. **Kartheek Gavini (Full-stack app lead):**
   Design and implement Express API endpoints (route structure, parameterization, error handling); build React page components and data-fetching logic; wire the end-to-end pipeline from browser to database; integration testing; deployment wiring (local + AWS later).